

Proyecto Integrador: Asistente Personal Local Basado en IA

Objetivo General

Diseñar e implementar un asistente personal basado en inteligencia artificial que opere de forma local en un sistema Linux, capaz de interactuar con archivos, documentos y recursos del sistema, proporcionando asistencia mediante lenguaje natural para tareas académicas, organizativas y de productividad.

El proyecto deberá priorizar la privacidad, la seguridad, la modularidad y la extensibilidad, evitando dependencias innecesarias de servicios en la nube.

Alcance del Proyecto

El asistente deberá ser capaz de:

- Mantener conversaciones en lenguaje natural.
 - Procesar documentos locales.
 - Buscar información dentro de archivos personales.
 - Generar resúmenes y análisis de documentos.
 - Interactuar con el sistema de archivos mediante herramientas controladas.
 - Mantener memoria persistente de información relevante.
 - Ejecutar tareas automatizadas bajo políticas de seguridad definidas.
-

Requerimientos Funcionales

RF-01: Modelo de lenguaje local

El sistema deberá utilizar un modelo de lenguaje ejecutado localmente.

Criterios de aceptación

- El modelo debe ejecutarse sin requerir conexión a Internet.
 - Debe aceptar consultas en español e inglés.
 - Debe poder integrarse mediante una API local.
-

RF-02: Interfaz de usuario

El sistema deberá proporcionar al menos una interfaz de interacción.

Opciones válidas:

- Terminal (CLI)
- Interfaz web local
- Interfaz gráfica de escritorio

Criterios de aceptación

- El usuario debe poder enviar consultas en lenguaje natural.
 - El historial de conversación deberá mantenerse durante la sesión.
-

RF-03: Gestión documental

El asistente deberá poder analizar documentos almacenados localmente.

Formatos mínimos requeridos:

- PDF
- TXT
- DOCX

Capacidades mínimas

- Extraer texto.
- Resumir documentos.
- Buscar palabras clave.
- Responder preguntas sobre el contenido.

Ejemplos

- "Resume este PDF."
 - "¿Qué dice este documento sobre OFDM?"
 - "Encuentra todas las referencias a LTE."
-

RF-04: Búsqueda semántica

El sistema deberá implementar recuperación de información basada en embeddings.

Capacidades

- Indexar documentos.
- Realizar búsquedas por significado y no únicamente por coincidencia textual.
- Recuperar fragmentos relevantes para alimentar al modelo de lenguaje.

Ejemplo

Consulta:

"¿Dónde hablo sobre modulación multicarrier?"

El sistema deberá localizar información relacionada incluso si el término exacto no aparece en el documento.

RF-05: Gestión de archivos

El asistente deberá contar con herramientas para interactuar con el sistema de archivos.

Funciones mínimas:

- Buscar archivos.
- Listar directorios.
- Leer archivos.
- Obtener metadatos.

Funciones opcionales:

- Mover archivos.
 - Renombrar archivos.
 - Organizar carpetas automáticamente.
-

RF-06: Memoria persistente

El sistema deberá almacenar información relevante entre sesiones.

Ejemplos:

- Proyectos activos.
- Preferencias del usuario.
- Ubicación de documentos frecuentes.

Restricción

La memoria deberá ser editable y eliminable por el usuario en cualquier momento.

RF-07: Arquitectura modular

Todas las capacidades del sistema deberán implementarse como módulos independientes.

Ejemplos:

- Módulo de documentos.
- Módulo de búsqueda.
- Módulo de memoria.
- Módulo de voz.
- Módulo de automatización.

El diseño deberá permitir agregar nuevas herramientas sin modificar el núcleo principal.

Requerimientos No Funcionales

RNF-01: Operación local

Toda la información procesada deberá permanecer en el equipo local.

Objetivo

Garantizar privacidad y autonomía operativa.

RNF-02: Rendimiento

El sistema deberá responder consultas comunes en tiempos razonables.

Objetivos recomendados:

- Conversación simple: menos de 10 segundos.
 - Consulta documental: menos de 20 segundos.
 - Búsqueda semántica: menos de 5 segundos.
-

RNF-03: Mantenibilidad

El código deberá:

- Seguir una estructura clara.
 - Incluir documentación.
 - Utilizar control de versiones mediante Git.
-

RNF-04: Escalabilidad

El diseño deberá permitir:

- Incorporar nuevos modelos.
 - Agregar nuevas herramientas.
 - Sustituir componentes sin reescribir el sistema completo.
-

Consideraciones de Seguridad

CS-01: Principio de mínimo privilegio

El asistente deberá operar con los permisos mínimos necesarios.

Justificación

Reducir el impacto de errores o comportamientos inesperados del modelo.

CS-02: Separación de modos de operación

El sistema deberá implementar al menos tres niveles de acceso:

Modo Lectura

Permite:

- Leer archivos.
- Buscar documentos.
- Consultar información.

No permite:

- Modificar archivos.

- Eliminar archivos.
- Ejecutar comandos.

Modo Escritura

Permite:

- Crear archivos.
- Renombrar archivos.
- Modificar documentos.

No permite:

- Eliminar archivos críticos.
- Ejecutar comandos arbitrarios.

Modo Administrador

Permite:

- Operaciones avanzadas.
- Automatizaciones del sistema.

Debe requerir confirmación explícita del usuario.

CS-03: Confirmación obligatoria

Toda acción potencialmente destructiva deberá solicitar aprobación explícita.

Ejemplos:

- Eliminar archivos.
- Sobrescribir documentos.
- Mover grandes cantidades de información.
- Ejecutar scripts.

Ejemplo

Solicitud:

"Borra la carpeta Descargas."

Respuesta esperada:

"Esta acción eliminará permanentemente 532 archivos. ¿Desea continuar? [Sí/No]"

CS-04: Lista blanca de directorios

El asistente únicamente podrá acceder a directorios autorizados.

Ejemplo:

```
/home/usuario/Documentos  
/home/usuario/Proyectos  
/home/usuario/Descargas
```

No deberá acceder a:

```
/etc  
/boot  
/usr
```

salvo autorización explícita.

CS-05: Registro de actividades

Todas las acciones deberán registrarse.

Información mínima:

- Fecha y hora.
 - Acción ejecutada.
 - Resultado.
 - Usuario que autorizó la acción.
-

CS-06: Protección contra comandos arbitrarios

El modelo de lenguaje no deberá ejecutar directamente texto generado por sí mismo.

Incorrecto

```
ejecutar(comando_generado_por_la_IA)
```

Correcto

Las acciones deberán pasar por una capa de validación que verifique:

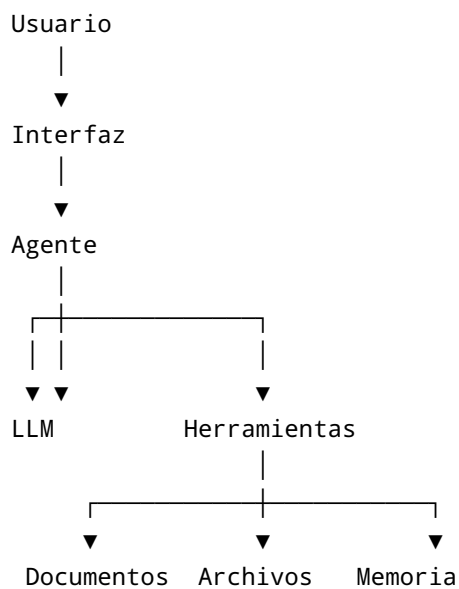
- Tipo de operación.
 - Parámetros.
 - Permisos.
 - Riesgo asociado.
-

CS-07: Recuperación ante fallos

Antes de realizar modificaciones importantes:

- Crear respaldos.
 - Mantener historial de cambios.
 - Permitir restauración.
-

Arquitectura Recomendada



Tecnologías Sugeridas

Modelo de lenguaje

- Ollama
- Qwen
- Llama

Programación

- Python

Recuperación semántica

- FAISS
- Chroma
- Qdrant

Procesamiento documental

- PyPDF
- pdfplumber
- python-docx

Voz (opcional)

- Whisper
- Piper

Entregables

1. Código fuente completo.
2. Documentación técnica.
3. Manual de usuario.
4. Video demostrativo.
5. Informe final describiendo:
6. Arquitectura implementada.
7. Tecnologías utilizadas.
8. Medidas de seguridad incorporadas.
9. Limitaciones encontradas.
10. Posibles mejoras futuras.

Criterios de Evaluación

Criterio	Porcentaje
Funcionalidad general	25%
Arquitectura y diseño	20%
Implementación de IA local	15%
Gestión documental y búsqueda semántica	15%
Seguridad y control de riesgos	15%
Documentación y calidad del código	10%

Observación Final

El objetivo principal del proyecto no es únicamente integrar un modelo de lenguaje, sino diseñar un sistema confiable que combine inteligencia artificial, recuperación de información, automatización y seguridad. Una implementación técnicamente sólida, segura y extensible será valorada por encima de una implementación con muchas funciones pero sin controles adecuados.